

ATENCAO — este PDF ainda tem campos por preencher.

Preencha nomes, RMs, usernames e os links do repositório e do vídeo em `docs/relatorio/RELATORIO-DE-ENTREGA.md` e gere de novo com `make relatorio`. O enunciado exige esses dados no entregável E3.1.

Relatório de Entrega — Tech Challenge Fase 5 (Hackathon)

SolidaryTech — plataforma de doações em AWS FIAP PosTech · DevOps & Arquitetura Cloud · Setembro de 2026

Este documento é a **fonte** do PDF exigido no entregável **E3**. Ele vive em Markdown para entrar em diff e ser revisado em PR como qualquer outro artefato; o PDF é gerado a partir dele com `make relatorio`. A instrução anterior era `pandoc ... -o .pdf`, que **não funciona sem um motor LaTeX instalado** (texlive, vários GB) — o comando falha com `pdflatex not found`, e a véspera da entrega é o pior momento para descobrir isso. O `make relatorio` não instala nada: converte para HTML com CSS de impressão e usa o Edge ou o Chrome que já existem na máquina, em modo headless. Saída em `docs/relatorio/RELATORIO-FASE5.pdf`.

1. Identificação (E3.1)

Nome	RM	Username GitHub
Leonardo Alves Freitas	rm369434	<code>freitasleoalves</code>
Antonio Eduardo Silveira Demarchi	rm370045	<code>demarchiworking</code>

Item	Link
Repositório (E3.2)	<code>https://github.com/DemarchiWorking/fiap-tc-5-solidarytech</code>
Vídeo (E3.2)	<code>a preencher</code>

2. Resumo executivo

A SolidaryTech é uma plataforma sem fins lucrativos que conecta ONGs, doadores e voluntários. Depois de ganhar destaque em rede nacional, passou a receber picos de acesso imprevisíveis, e a diretoria estabeleceu quatro exigências: **as doações não podem parar, o custo precisa ser justificado, os incidentes precisam ser previstos e os acordos com as ONGs precisam ser claros.**

Esta entrega responde às quatro com número medido, não com intenção.

Exigência da diretoria	Resposta	Evidência
Se a nuvem cair, as doações não param	RTO 1 h · RPO 15 min, com DR cross-region testado	\$6
O custo precisa ser justificado	US\$ 202/mês, com custo por serviço e 5 otimizações quantificadas	\$5
Os incidentes precisam ser preditivos	AIOps + burn rate de error budget · MTTR de ~10 h para ~10 min	\$4, \$7
SLO/SLA claros com as ONGs	3 SLIs medidos · SLO 99,9% · SLA 99,5% com crédito de serviço	\$4

Ambiente: AWS Academy Learner Lab

Todo o projeto roda no ambiente gratuito que a FIAP disponibiliza. Isso **não é um detalhe de execução — é a restrição que moldou a arquitetura**. O Learner Lab bloqueia a criação de IAM roles, users e OIDC providers, o que elimina IRSA, `eksctl`, o AWS Load Balancer Controller e a federação OIDC do GitHub Actions.

Cada contorno está documentado em ADR, com o desenho de produção ao lado. Nenhuma restrição foi escondida.

3. Fundação DevOps — Fases 1 a 4 (Requisito obrigatório)

#	Requisito	Como foi atendido
F0.1	Docker e Kubernetes	Dockerfiles multi-stage com estágio de teste, usuário não-root por UID e <code>HEALTHCHECK</code> . Go em <code>distroless</code> ; Python sem compilador na imagem final. Deploy em EKS
F0.2	IaC (Terraform)	21 arquivos <code>.tf</code> : backend S3+DynamoDB, 8 módulos, 2 ambientes. Cluster, bancos, mensageria e rede — 100% por código
F0.3	CI/CD DevSecOps	Pipeline reutilizável: <code>lint test</code> → <code>sonar</code> + <code>build-scan-push</code> → <code>update-gitops</code> . Trivy em 2 camadas (SCA + imagem), SBOM CycloneDX, <code>gitleaks</code>
F0.4	GitOps	ArgoCD com App-of-Apps → ApplicationSet. <code>selfHeal</code> e <code>prune</code> ligados. Um único <code>kubectl apply</code> em todo o projeto
F0.5	Observabilidade e APM	Prometheus, Grafana, Loki (S3), dois OTEL Collectors. New Relic com Distributed Tracing atravessando o SQS

Comparativo completo das três entregas: `docs/02-arquitetura/evolucao-v3-v4-v5.md`

Melhorias sobre a entrega da Fase 4

Lacuna da Fase 4	Correção nesta entrega
Sem métrica de latência — só contador; SLO de latência era incalculável	Histograma <code>solidary.http.server.duration</code> com nome, unidade e buckets idênticos em Go e Python
SLOs "propostos, não implementados"	<code>PrometheusRule</code> com burn rate multi-janela e error budget renderizado em painel
Segredos reais versionados em texto puro (senha do Postgres, chave do Service Bus, <code>DD_API_KEY</code>)	Senha no Secrets Manager ; credencial de nuvem via IMDS — não existe como segredo. <code>gitleaks</code> no CI
IP público do LB fixado no <code>values.yaml</code> — quebrava a cada recriação	Hostname vindo do output do Terraform
Sem <code>startupProbe</code>, sem <code>PodDisruptionBudget</code>	Ambos em todos os workloads
Fila sem consumidor — trace terminava no produtor	<code>volunteer-worker</code> acrescentado; trace ponta a ponta real

Correções no código fornecido pelo enunciado

O `donation-service` original **não compila** (`strconv` e `fmt` importados e não usados — em Go isso é erro, não aviso). O `volunteer-service` respondia **500 permanente** em `GET /volunteers/<ngo_id>`, porque o DynamoDB retorna números como `decimal.Decimal` e o encoder JSON do Flask não serializa esse tipo.

22 defeitos corrigidos, todos com teste de regressão. Tabela completa em `services/README.md`.

4. Seção SRE (E3.3)

Evidência visual obrigatória: definição formal de SLI, SLO e SLA do serviço de doações.

Documento completo: [docs/03-sre/sli-slo-sla.md](#)

SLIs e SLOs do `donation-service`

#	SLI	Golden Metric	SLO	Error budget
1	Proporção de requisições não-5xx	Errors	99,9% / 7 d	0,1%
2	Proporção servida em < 300 ms	Latency	99% / 7 d	1%
3	Eventos processados em < 60 s	Saturation	99,5% / 7 d	0,5%

O enunciado exige no mínimo dois. Entregamos três — e o terceiro é o que fecha um ponto cego real: a doação é confirmada **antes** do consumo da fila, então o worker poderia estar parado há horas com os outros dois SLIs verdes.

SLA com as ONGs parceiras

	SLO (interno)	SLA (contratual)
Disponibilidade	99,9%	99,5% ao mês
Latência	99% < 300 ms	95% < 1 s
Consequência	Congelamento de deploy	Crédito de serviço

O SLA é mais frouxo que o SLO de propósito: os ~3,2 h/mês de folga são o que absorve um incidente ruim sem quebrar contrato.

Política de error budget — automatizada

Consumido	Ação
> 80%	Congelamento de deploys não críticos (hotfix e self-heal seguem)
100%	Congelamento total até o post-mortem concluído

Aplicado removendo `syncPolicy.automated` do Application — o ArgoCD para de sincronizar, mas o self-heal continua funcionando. Congelar mudanças **não pode** significar congelar a capacidade de mitigar.

O error budget em ação — um achado real do teste de carga

O ciclo abaixo não é ilustrativo. Aconteceu neste ambiente, e é a melhor evidência de que os SLIs fazem trabalho de verdade.

Detecção. Depois da primeira carga (8.755 eventos), o SLI de frescor acusou:

Eventos com lag ≤ 60 s	6.137 — 70,1 %
Eventos com lag > 60 s	2.618 — 29,9 %
Lag médio	39,7 s
Error budget restante	-42,38 (estourado em mais de 40×)

Os outros dois SLIs estavam verdes. Só o de frescor viu — e é exatamente o ponto cego que ele foi criado para cobrir: a doação é confirmada **antes** do consumo da fila.

Diagnóstico. Duas causas somadas, nenhuma delas produzindo erro:

1. **O EKS não instala o `metrics-server`, e ninguém instalou.** Sem a API

`metrics.k8s.io`, um HPA não falha — fica em `<unknown>` e nunca escala. Os três HPAs do projeto estavam declarados, criados, listados, e eram **decorativos**. `kubectl top` também não respondia.

1. **O `volunteer-worker` não tinha HPA nenhum.** Rodava fixo em 1 réplica com

50 m de CPU. O produtor escalava (na intenção); o consumidor, não.

O script do k6 tem um estágio comentado como "*PICO — dispara o HPA*". O pico chegava, e nada disparava.

Por que nenhum gate pegou. `kustomize build`, `kubeconform`, `terraform validate` e os cinco gates locais validam a **forma**. Nenhum deles executa um cluster, e um HPA sintaticamente perfeito que nunca escala passa por todos. Foi preciso rodar carga contra o ambiente real para o defeito aparecer.

Ação. Pelo GitOps, em dois commits: `metrics-server` como addon (onda de sync -2, antes de tudo) e HPA próprio para o worker.

Recuperação. Assim que a Metrics API subiu, o HPA do worker leu **194 %** de utilização e escalou sozinho em 81 segundos. Na carga seguinte:

	Antes	Depois
Réplicas do worker	1 (fixo)	1 → 6 (HPA)
Eventos fora do alvo	29,9 %	0,1 % (4.862 de 4.867)
<code>kubectl get hpa</code>	<code>cpu: <unknown>/70%</code>	<code>cpu: 1%/70%</code>

O que o pico revelou de quebra. O HPA chegou a **404 %** de utilização — o request de 50 m estava quatro vezes abaixo do uso real (~200 m). Isso não mata o pod, porque o *limit* não era atingido; o que se degrada em silêncio é o **agendamento**: o scheduler acreditava que quatro workers custavam 200 m quando custavam 800 m. Corrigido para 100 m de request — o dobro do regime e metade do pico, porque request igual ao pico faria a utilização nunca passar de 100 % e o HPA nunca escalar.

O orçamento de 7 dias não voltou ao verde, e não deveria. A taxa instantânea está no alvo; o error budget leva dias para cicatrizar. É essa a função dele: lembrar do que aconteceu depois que o gráfico já voltou ao normal.

Evidência: `elasticidade-e-frescor.txt`

MTTR (F1.3)

Etapa	Sem a stack	Com a stack
Detecção	~6 h (usuário reclama)	~2 min (burn rate)
Diagnóstico	~4 h (logs por pod)	~5 min (<code>trace_id</code> : APM ↔ Loki)
Mitigação	minutos, se houver alguém	~90 s (self-heal)
Total	~10 h	~10 min

Procedimento do drill: `mttr-chaos-drill.md`.

Evidências: (inserir prints de `docs/07-evidencias/`)

5. Seção FinOps (E3.4)

Evidência visual obrigatória: análise de custos mensais (Forecast) e evidências das tags aplicadas.

Documento completo: [docs/04-finops/README.md](#)

Forecast — US\$ 201,94/mês

Item	US\$/mês
EKS control plane	72,00
3 × <code>t3.medium</code>	90,00
RDS <code>db.t3.micro</code>	12,90
NLB	16,20
EBS (nós + PVCs)	8,24
S3 + DynamoDB + SQS + ECR + CloudWatch	2,60
Total	201,94

Tags obrigatórias

`Project=SolidaryTech` · `Environment=Production` · `CostCenter=NGO-Core` (+ `ManagedBy`, `Owner`, `Phase`)

Aplicadas por `default_tags` no provider. **A armadilha:** `default_tags` **não alcança** as EC2 nem os volumes de um managed node group — e são eles que dominam a fatura. Resolvido com **launch template** e `tag_specifications`.

Recomendações quantificadas

O enunciado pede pelo menos uma. São cinco.

#	Recomendação	Economia
1	Desligar o ambiente fora de uso (<code>make lab-down</code>)	US\$ 155/mês (77%)
2	<code>gp2</code> → <code>gp3</code> (aplicado)	~20% do EBS
3	<code>Scan</code> → <code>Query</code> no DynamoDB (GSI já criado)	60–90% da leitura
4	Spot Instances — não aplicável: o lab só libera On-Demand	(US\$ 60/mês em produção)
5	VPC Endpoints de gateway (aplicado, gratuitos)	elimina custo de NAT p/ S3 e DynamoDB

Evidências: (print do Tag Editor + dashboard FinOps + tabela de rightsizing)

6. Seção Segurança e DR (E3.5)

Evidência visual obrigatória: documento de PCN (com RPO e RTO) e explicação da estratégia de DR.

Documento completo: docs/06-dr-pcn/pcn.md

RTO e RPO

Serviço	Criticidade	RTO	RPO
donation-service	Crítica	1 h	15 min
ngo-service	Alta	4 h	24 h
volunteer-service	Média	8 h	24 h

Compromisso assimétrico de propósito: uma doação perdida é perda financeira direta; um cadastro de ONG perdido é um formulário reenviado.

Estratégia de DR — as duas opções

O enunciado pede A **ou** B. Entregamos as duas: elas protegem contra falhas diferentes.

Opção A — Velero: backup de manifestos e volumes para bucket S3 em us-west-2, região diferente da do cluster. Horário (aplicações) e diário (cluster). Autentica pelo **IMDS do nó** — a instalação padrão exigiria IRSA, bloqueado no lab.

Opção B — Warm standby: make dr-up sobe a região espelho. O ambiente de DR **não redefine nada** — chama os mesmos módulos com outra região e capacidade reduzida. É isso que prova a modularização.

Débitos de segurança declarados

Consequências do ambiente da faculdade, com o desenho correto documentado: sem Multi-AZ · sem IRSA · sem TLS/WAF · sem CMK no etcd · credencial estática no CI (OIDC bloqueado) · nós em subnet pública (decisão de custo revertível por uma variável).

Evidências: (print do restore do Velero + make dr-plan limpo)

7. Seção ITSM e AIOps (E3.6)

Evidência visual obrigatória: desenho do ciclo de vida de incidentes.

Documento completo: [docs/05-itsm-aiops/README.md](#)

Ciclo de vida do incidente

```
DETECÇÃO —┬─ determinística (burn rate de SLO)
              └─ preditiva (AIOps: anomalia comportamental)
              v
TRIAGEM (page / ticket)
              v
NOTIFICAÇÃO — PagerDuty · ChatOps · self-heal ← em PARALELO
              v
MITIGAÇÃO AUTOMÁTICA (rollout restart c/ allowlist)
              v
INVESTIGAÇÃO (trace_id: APM ↔ Loki) ou ESCALONAMENTO
              v
RESOLUÇÃO
              v
POST-MORTEM BLAMELESS (48 h, obrigatório em todo page)
              v
COMUNICAÇÃO (ONGs · diretoria · time)
              └─ realimenta a detecção
```

Diagrama completo em [docs/05-itsm-aiops/README.md](#).

AIOps

New Relic Applied Intelligence — Anomaly Detection, Correlated Incidents e Golden Signals automáticos.

A escolha sobre o Datadog está no [ADR-004](#): o trial do Datadog dura 14 dias e o hackathon dura 2 meses; seu free tier permanente **não inclui APM**, que é o requisito.

Evidências: *(print da anomalia detectada + execução do self-heal)*

8. Como reproduzir

```
make bootstrap      # 1x por conta: bucket de state
make lab-up         # infraestrutura (~20 min)
make configurar-repo # ajusta o GitOps para a conta; commit + push
make deploy         # ArgoCD assume e sincroniza tudo
make carga          # gera tráfego para os painéis
make lab-down       # AO FIM DE CADA SESSÃO
```

Detalhes em [README.md](#) e [infra/README.md](#).

9. Rastreabilidade

Matriz completa requisito → critério → artefato → evidência: [docs/01-requisitos-e-criterios-de-aceitacao.md](#)

40 requisitos mapeados (28 do enunciado + 12 entregáveis), com classificação de risco de dedução de pontos.